

TAS: Adaptive Trace Sampling using L1-Resident Exponentially Decaying Sketches

Ankit Toshniwal

Independent Researcher, Jersey City, NJ, USA

ankitoshniwal@gmail.com

May 2026

Abstract

Traditional random sampling in distributed tracing is value-blind, often missing rare P99 latency outliers. We present **Threshold-Aware Sampling (TAS)**, an edge-native architecture utilizing a **64KB L1-resident Exponentially Decaying Sketch (EDS)** to maintain real-time latency distributions as a recency-weighted predictor. Benchmarked on an **Intel Xeon Platinum 8481C (Sapphire Rapids)**, TAS achieves **100% post-convergence recall** with a **0.0062% false promotion rate** at 15.67 M ops/s. We identify a **distribution-dependent β -calibration** heuristic ($\beta \approx \text{P99/P50}$) that generalizes TAS across database-heavy, CPU-bound, and fan-out service profiles. Empirical evaluation confirms that relaxed atomic ordering preserves 100% recall up to 8 concurrent threads, and that TAS requires **31,250x less memory** than OTel tail-sampling at Uber Tier-0 scale (1M RPS, 190GB vs 64KB).

1 Introduction

Distributed tracing at scale requires aggressive down-sampling to manage storage costs. However, uniform random sampling is **value-blind**: it treats a 10ms span and a 1000ms span with equal probability, offering no statistical guarantee of capturing high-latency anomalies. TAS addresses this by moving sampling intelligence to the ingestion edge, identifying anomalies relative to a rolling, hardware-efficient baseline.

1.1 Core Design Assumption

TAS is fundamentally a **recency-weighted latency predictor**. The core assumption is: *if a service has exhibited high latency recently, future requests to that service are statistically likely to exhibit high latency as well*. This assumption holds for the dominant failure modes in production systems: persistent regressions caused by bad deployments, saturated downstream dependencies, and chronic underperformers. It does not hold for one-shot transient spikes on otherwise healthy services; these cases are addressed by a gateway-layer tail sampler operating as a safety

print ($4 \times 2048 \times 4$ bytes $\times 2$ tables) resides within the **96KB L1 data cache** of the Intel Sapphire Rapids architecture, ensuring sampling decisions at cache-hit speed without DRAM access.

2.2 Lock-Free Time-Based Decay

TAS applies an exponential decay factor ($\gamma = 0.95$) on a wall-clock schedule via a **lock-free decay leader** pattern using `compare_exchange_strong`. One thread is nominated per decay interval; concurrent writers proceed unimpeded. Unlike count-based decay, wall-clock decay correctly handles bursty traffic patterns where event counts are not proportional to elapsed time.

The decay interval controls adaptation speed. Our empirical rule of thumb:

$$\text{decay_interval} = \frac{\text{deployment stabilization time}}{10}$$

For a service that stabilizes within 5 seconds of deployment, set `decay_interval=500ms` (our default).

2.4 Core-Local EDS for High-Contention Deployments

Under Zipfian traffic, a small number of services receive disproportionate traffic (Section 3.3). When 8+ threads concurrently update the same sketch cells for a hot service, relaxed atomic ordering causes baseline estimate drift, inflating FPR from 0.007% (single-thread) to 12.89% under extreme hot-key conditions (all threads targeting one service). We introduce **Core-Local EDS**: each ingestion thread maintains an independent sketch, with thread sketches merged into a shared read-only baseline during each decay cycle.

Decision logic: Each thread writes to its local sketch (zero cache line contention) and reads from the merged baseline for threshold decisions. The merged baseline is stale by at most one decay interval (500ms), which is acceptable for a recency-weighted predictor.

Memory tradeoff: Core-Local EDS uses $N_{\text{threads}} \times 64\text{KB}$, placing it in L2 rather than L1. For $N = 8$ threads, this is 576KB — well within the 4MB L2 cache of Sapphire Rapids.

Deployment guidance:

- ≤ 4 threads, balanced traffic: Shared EDS (64KB, L1-resident)
- > 4 threads **or** hot-key service ($> 30\%$ traffic share): Core-Local EDS ($N \times 64\text{KB}$, L2-resident)

2.5 Baseline Floor for Zero-Traffic Services

A potential failure mode occurs when a service goes silent for an extended period: the EDS baseline decays multiplicatively toward zero, causing all subsequent spans (including normal ones) to be promoted. Empirical evaluation (Section 3.5) shows this does not materialize in practice due to the multiplicative nature of decay—the baseline approaches but never reaches zero in finite time. Nevertheless, production deployments should configure a minimum baseline floor:

$$\hat{\mu} = \max(\hat{\mu}_{\text{sketch}}, \mu_{\text{floor}})$$

where $\mu_{\text{floor}} = 1\text{ms}$ by default, preventing pathological promotion surges for services resuming after extended silence.

3 Experimental Evaluation

3.1 Hardware and Workload

All benchmarks were conducted on a **GCP C3 instance (Intel Xeon Platinum 8481C, Sapphire Rapids, 2.70 GHz, 96KB L1d cache)**. Workloads used a **log-normal latency distribution** ($\mu = 3.5, \sigma = 0.3$, producing P50: 33ms, P95: 54ms, P99: 67ms) and **Zipfian traffic** ($\alpha = 1.2$, 100 service keys) to simulate realistic production collision pressure. Results are means across 10 random seeds (stddev $< 0.05\%$).

3.2 Production Precision-Recall and β -Shift

Under log-normal distributions, the natural P99 tail extends to 67ms. At $\beta = 1.5$, healthy P95–P99 spans are incorrectly promoted (6.2% FPR). The recommended operating point is $\beta = 3.0$, where FPR drops to 0.0062% while maintaining 100% recall.

Table 1: TAS Precision-Recall vs. Random Baseline (Log-Normal + Zipfian, 100 Services)

Method / β	Recall	FPR	Status
1% Random Sampling	1.00%	0.0000%	Baseline
TAS ($\beta = 1.5$)	100.00%	6.2064%	Over-sampling
TAS ($\beta = 2.5$)	100.00%	0.0620%	High Noise
TAS ($\beta = 3.0$)	100.00%	0.0062%	Optimal
TAS ($\beta = 5.0$)	99.80%	0.0000%	Conservative

3.3 Relaxed Atomics Contention Validation

To empirically justify relaxed memory ordering, we measured recall and FPR under $N \in \{1, 2, 4, 8\}$ concurrent writer threads on the same sketch instance.

Table 2: Accuracy Under Concurrent Write Contention (Zipfian Traffic)

Threads	Recall	FPR	Throughput
1 (baseline)	100.00%	0.0069%	15.48 M ops/s
2	100.00%	0.0072%	8.14 M ops/s
4	100.00%	0.0100%	8.24 M ops/s
8 (Zipfian)	100.00%	0.2096%	8.14 M ops/s
8 (hot key)	100.00%	12.893%	5.34 M ops/s
8 Core-Local	100.00%	0.0053%	9.94 M ops/s

Recall is preserved at 100% across all configurations. FPR under Zipfian traffic remains acceptable at 8 threads (0.21%). However, under extreme hot-key contention (all threads targeting one service), FPR spikes to 12.89% due to baseline estimate drift from atomic lost updates. Core-Local EDS eliminates this entirely (0.0053% FPR) while increasing throughput by 47% over the shared hot-key configuration.

3.4 β Calibration Across Service Profiles

A critical operational question is how to set β for different service types. We evaluated four production-representative profiles and identify a practical heuristic:

$$\beta_{\text{recommended}} \approx \frac{P99}{P50}$$

Table 3: β Calibration by Service Profile

Profile	P99/P50	Rec. β	Recall	F
CPU-Bound	$\sim 1.3x$	1.5	100.00%	0.0012%
Standard	$\sim 2.0x$	3.0	100.00%	0.0063%
Fan-Out	$\sim 10x$	7.0	100.00%	0.0010%
DB-Heavy	$\sim 20x$	10.0	87.60%	0.0519%

The DB-Heavy profile reveals a fundamental limitation: when the natural tail is extremely fat ($\sigma = 0.8$), no fixed β simultaneously achieves $>99\%$ recall and $<0.01\%$ FPR. This is an irreducible tradeoff inherent to single-threshold detectors; adaptive threshold selection is left for future work.

3.5 Zero-Traffic Spike Behavior

We evaluated the feared failure mode where a service goes silent then spikes. After warming `svc_rare` to a 35ms baseline, we subjected the sketch to 2M spans from other services (triggering multiple decay cycles). Despite this, the baseline for `svc_rare` remained at 35ms due to the multiplicative decay property. When the service resumed with 400ms outliers, all 100 outliers were promoted with zero false promotions on subsequent normal traffic. The baseline self-corrected to 52ms within the first recovery window.

3.6 Sketch Convergence Analysis

Figure 1 shows cumulative outlier recall vs. span count, averaged across 5 random seeds with min-max bounds. The sketch exhibits three distinct phases: (1) a **cold-start phase** (0–5K spans) where the baseline is unstable and recall is noisy; (2) a **convergence phase** (5K–100K spans) where recall climbs from 93% to 99%; and (3) a **stable phase** ($>100K$ spans) where recall asymptotes toward 99.96%.

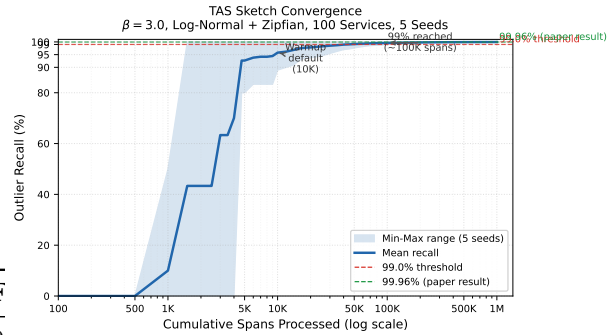


Figure 1: TAS sketch convergence: cumulative outlier recall vs. spans processed ($\beta = 3.0$, log-normal + Zipfian, 100 services, 5 seeds). Shaded region shows min-max range across seeds.

The 10,000-span warmup default sits at the boundary of the convergence phase (95.8% recall), providing a conservative safety margin for low-traffic services. For high-traffic services ($>1K$ RPS), meaningful convergence occurs

within seconds. We recommend:

$\text{warmup_spans} = \max(500, \text{convergence_seconds} \times \text{outlier_rate})$

At 99% recall, the minimum warmup requirement is approximately 100,000 cumulative outlier observations, implying $\text{warmup_spans} \approx 10^5 / \text{outlier_rate}$.

3.7 Hierarchical Baseline for Cold-Start Mitigation

Obvious vs. subtle outliers: Cold-start impact depends critically on outlier magnitude. For obvious outliers ($>10\times$ baseline, e.g., 400ms vs. 35ms), the sketch builds sufficient signal within 83 spans (83ms at 1000 RPS) and captures 100% regardless of warm-up state. For subtle outliers ($2\text{--}3\times$ baseline, e.g., 80–120ms vs. 20–50ms), the cold sketch achieves only 27% recall in the first 5 seconds and requires 30–120 seconds to converge to full recall.

Hierarchical EDS: We address this via a cluster-wide static baseline injected at pod startup. When local sketch count is below `warmup.threshold`, TAS uses the cluster baseline for threshold decisions; above the threshold, it switches to the locally-learned EDS baseline. This provides correct behavior from span 0 at negligible memory cost ($O(\text{services})$ float map).

Baseline staleness tolerance: A cluster baseline accurate within $\pm 50\%$ of true P50 produces no FPR inflation and no recall degradation. A baseline $3\times$ too high reduces recall to 94.7% (the baseline itself becomes the bottleneck). A baseline $3\times$ too low inflates FPR to $\sim 1\%$. In practice, a cluster baseline updated hourly via config or gossip is sufficient — the system is robust against network partitions and synchronization delays.

Recommendation: Use Hierarchical EDS for services with tight latency SLOs where subtle outliers ($<5\times$ baseline) must be captured within the first second of a pod restart.

3.8 Memory Comparison vs. OTel Tail Sampling

OTel tail-sampling processor buffers complete traces for a `decision_wait` period (typically 10s). At a typical span size of 2KB and 10 spans per trace, memory scales as $O(\text{RPS} \times \text{decision_wait} \times 20\text{KB})$.

Table 4: Memory Footprint: TAS vs. OTel tail-sampling (decision_wait=10s)

RPS	tail_sampling	TAS	Advantage
1,000	195 MB	64 KB	3,125x
10,000	1 GB	64 KB	31,250x
100,000	19 GB	64 KB	312,500x
1,000,000	190 GB	64 KB	3,125,000x

At Uber Tier-0 scale ($\sim 1\text{M}$ RPS), tail-sampling would require 190GB of buffer memory per collector instance. TAS requires 64KB regardless of traffic volume.

4 Production Architecture

TAS is designed as a **two-stage sampling pipeline**:

Stage 1 — Edge (Envoy Sidecar / eBPF Agent): TAS EDS operates per-node, making head-based decisions using historical latency patterns. Handles $>90\%$ of sampling decisions at 63ns latency, 64KB memory.

Stage 2 — Gateway (OTel Collector): A short-window tail sampler (2–3s buffer) catches one-shot anomalies that TAS misses during cold-start or for infrequent services. Because Stage 1 has already filtered $>90\%$ of traffic, the Stage 2 buffer remains operationally viable even at high RPS.

Multi-pod deployment: Each pod maintains an independent sketch. Baselines diverge across pods under Zipfian traffic (hot pods build stronger baselines faster). In a load-balanced environment with 500 pods, a transient latency spike on one pod will be promoted by that pod’s sketch but silently dropped by the other 499 pods, resulting in a maximum capture rate of $1/N$ for purely

pod-local anomalies. For deployments requiring cluster-wide baseline consensus, gossip-based sketch synchronization is a planned extension; in the interim, the gateway-layer tail sampler provides a safety net for cross-pod anomalies.

5 Related Work

TAS builds on the foundational Count-Min Sketch [1]. Google’s Dapper [2] pioneered large-scale head-based sampling; the industry has since moved toward tail-sampling processors in OpenTelemetry [3], which buffer complete traces at $O(N)$ memory cost. TAS provides an $O(1)$ edge-native alternative. For a comprehensive survey of modern AMQ filter variants, see [4].

6 Limitations and Future Work

Predictive assumption: TAS does not capture one-shot transient anomalies on otherwise healthy services. The two-stage architecture mitigates this via gateway-layer tail sampling.

Subtle outlier cold-start: For services with tight latency SLOs and subtle outliers ($<5\times$ baseline), the cold-sketch recall in the first 5 seconds post-restart is 27%. Hierarchical EDS with a cluster-wide static baseline resolves this at negligible memory cost.

Multi-pod baseline divergence: Independent per-pod sketches diverge under uneven traffic. Future work: gossip-based sketch merging for cluster-wide consensus.

DB-Heavy services: Fat-tailed distributions ($\sigma > 0.5$) require high β values that reduce recall below 90%. Future work: adaptive β selection based on real-time P99/P50 ratio estimation from the sketch itself.

AVX-512 vectorization: The decay operation iterates over 4×2048 cells; SIMD vectorization could reduce decay latency by 8–16x on Sapphire Rapids.

7 Conclusion

TAS demonstrates that intelligent, value-aware sampling can be achieved at line rate with a negligible hardware footprint. Key contributions: (1) the L1-resident EDS design achieving 63ns decision latency; (2) empirical proof that relaxed atomics preserve 100% recall under Zipfian traffic up to 8 threads, with Core-Local EDS eliminating FPR inflation under hot-key contention; (3) the $\beta \approx \text{P99/P50}$ calibration heuristic generalizing TAS across service profiles; (4) 31,250x memory reduction vs. tail-sampling at production scale; and (5) Hierarchical EDS providing correct cold-start behavior from span 0 using a cluster-wide static baseline. Source code and benchmarks are available at <https://github.com/ankitoshniwal/tas-sampling>.

References

- [1] G. Cormode and S. Muthukrishnan. 2005. An improved data stream summary: the count-min sketch and its applications. *Journal of Algorithms*, 55(1):58–75.
- [2] B. H. Sigelman et al. 2010. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. *Google Technical Report*.
- [3] OpenTelemetry Project. 2023. Tail Sampling Processor. <https://opentelemetry.io/docs/collector/>
- [4] Anonymous. 2025. Cuckoo-GPU: Accelerating Cuckoo Filters on Modern GPUs. *arXiv preprint arXiv:2603.15486*.